

NGS2026 — Report OCR Targhe: confronto completo delle tecnologie

New Generation Sensors · Edge AI Pipeline · Maggio 2026

Sommario esecutivo

Questo documento unifica i risultati di tutti i benchmark OCR condotti finora sul progetto NGS2026, su un unico dataset di riferimento: **European License Plates (735 crop di targhe europee miste, formato e qualità eterogenei)**. L'obiettivo è confrontare in modo coerente le tecnologie OCR disponibili e identificare il candidato migliore per la pipeline di lettura targhe operativa su Raspberry Pi 5.

Risultato chiave: la tecnologia OCR specializzata `fast-plate-ocr` (modello `cct-xs-v1-global`) raggiunge un'accuratezza di **86.0% exact match** con **21 ms per crop**, contro il **16.7% / 1000 ms** della soluzione baseline EasyOCR — un salto di oltre 5× in accuratezza e ~50× in velocità, su hardware Raspberry Pi 5 senza GPU.

1. Setup dei benchmark

1.1 Dataset di riferimento

European License Plates: 735 crop di targhe di vari paesi europei (Italia, Francia, Spagna, Germania, Portogallo, Irlanda, ecc.), con formati e font eterogenei. La ground truth è derivata dal nome del file (es. `THW82986.png` GT `THW82986`).

Caratteristiche rilevanti del dataset:

- Crop di targa già ritagliati (no detection necessaria)
- Tante targhe presentano **rumore di contesto**: codici paese sulla banda blu UE, scritte di concessionari, prefissi regionali, numeri di telefono visibili sulla cornice della targa
- Formati eterogenei: oltre allo standard italiano `AA000AA`, presenti formati a 6-9 caratteri di paesi diversi

1.2 Hardware

- Raspberry Pi 5 (16 GB RAM)
- Esecuzione su CPU (no GPU disponibile sulla RPi5)
- ONNX Runtime per i modelli compatibili
- Senza accelerazione hardware Hailo per OCR (Hailo-8 è dedicato a YOLO detection nella pipeline)

1.3 Metriche

- **Exact match accuracy**: percentuale di predizioni identiche carattere per carattere alla GT
- **Avg CER (Character Error Rate)**: Levenshtein distance / lunghezza GT, mediato sul dataset
- **Tempo medio per crop** in millisecondi (CPU only)

1.4 Convenzioni di pre-processing

Per ciascuna tecnologia sono state valutate, quando rilevante, varianti di pre-processing dell'immagine prima dell'OCR (raw, grayscale, CLAHE, ecc.). I dettagli sono descritti nei rispettivi capitoli.

Nessun post-processing applicato in questo documento: i numeri qui riportati misurano la **performance grezza** dei modelli OCR.

2. EasyOCR

2.1 Cos'è

EasyOCR è una libreria Python open-source per riconoscimento ottico di caratteri (OCR) generale, basata su deep learning. Combina due reti neurali: **CRAFT** per la detection di regioni testuali e **CRNN** (CNN + Bi-LSTM) per il riconoscimento del testo. Supporta 80+ lingue e gira su CPU o GPU.

Punti tecnici rilevanti:

- Accetta in input immagini di **qualsiasi formato** (BGR, RGB, grayscale): la libreria gestisce internamente le conversioni
- Modello relativamente pesante (~70 MB)
- Pensato per OCR generico, **non specializzato per targhe**

- Permette `allowlist` per restringere il set di caratteri attesi (utile per targhe: solo A-Z + 0-9)

2.2 Risultati sul dataset European License Plates

Variante	Exact match	Avg CER	Tempo medio
EasyOCR raw (baseline, run completo 735)	16.73%	0.698	1612 ms

Interpretazione del basso exact match: l'analisi degli errori ha rivelato che il problema principale di EasyOCR su questo dataset **non è la qualità della lettura dei caratteri**, ma la **delimitazione del testo della targa** rispetto al contesto. In circa il 36% dei casi la targa viene letta correttamente, ma circondata da rumore aggiunto (codici paese, nomi rivenditori, numeri di telefono delle concessionarie). Questo si manifesta come CER medio elevato (predizioni più lunghe della GT) ma in realtà l'OCR ha "visto" la targa giusta.

2.3 Pre-processing dell'immagine

Sono state testate sistematicamente 8 varianti di pre-processing applicate prima di EasyOCR, su un primo screening di 50 immagini.

Screening 50 immagini:

#	Variante	Exact	Avg CER	Median CER	Tempo
1	baseline_raw	16.0%	1.086	0.333	1197 ms
2	gray_clahe	16.0%	1.102	0.354	1010 ms
3	grayscale	12.0%	1.079	0.333	1008 ms
4	gray_clahe_sharpen	12.0%	1.087	0.286	1097 ms
5	upscale_gray_clahe	12.0%	1.227	0.429	12275 ms
6	adaptive_threshold	8.0%	1.155	0.375	1113 ms
7	gray_clahe_threshold	4.0%	1.198	0.500	1426 ms
8	baseline_resize_4x	2.0%	1.250	0.464	11583 ms

Run completo 735 immagini sulle 4 migliori varianti:

Variante	Exact raw	Substring match	Avg CER	Tempo
----------	-----------	-----------------	---------	-------

grayscale	17.41%	36.19%	0.697	1239 ms
baseline_raw	16.73%	35.24%	0.698	1612 ms
gray_clahe	15.65%	34.29%	0.683	1173 ms
gray_clahe_sharpen	13.88%	32.38%	0.679	1179 ms

Conclusioni preprocessing:

- Le tecniche di pre-processing dell'immagine **non sono la leva principale** per migliorare EasyOCR su questo dataset
- La variante migliore (grayscale) supera la baseline di soli 0.7 punti percentuali
- Le tecniche aggressive (binarizzazione, threshold) **degradano** le performance: EasyOCR è addestrato su immagini naturali
- L'upsampling 4× è controproducente: triplica il tempo di esecuzione senza migliorare l'accuratezza
- La metrica **substring match (36%)** rivela il potenziale teorico massimo recuperabile con post-processing: nel 36% dei casi la targa è letta correttamente ma "sporca" di contesto

2.4 Errori sistematici di EasyOCR

L'analisi degli errori carattere-per-carattere ha mostrato confusioni ricorrenti:

Carattere GT	Predetto come	Occorrenze (su 735)
0 (zero)	O (lettera)	43
4	L	19
7	Z	15
D	0	11
M	H	11
1	I	11
W	M	9

Sono confusioni geometriche di forma del carattere su font targa, tipiche di un OCR generico non specializzato.

3. Gemini API (Google Cloud)

3.1 Cos'è

Gemini è la famiglia di modelli multimodali di Google, accessibile via **API cloud** (Vertex AI). I modelli accettano immagini e producono testo come risposta. Per OCR su targhe, è sufficiente passare l'immagine con un prompt tipo "Read the license plate" e il modello ritorna il testo.

Caratteristiche rilevanti:

- **Non gira on-device:** richiede connessione di rete e chiamata API
- Costo per chiamata (~0.001-0.01 €/chiamata a seconda del modello)
- Tre versioni testate: **2.5 Flash-Lite, 2.5 Flash, 2.5 Pro**
- Disabilitato il "thinking mode" su Flash/Flash-Lite per evitare chain-of-thought leakage nell'output

3.2 Risultati sul dataset European License Plates

Modello	Exact match	Avg CER	Tempo medio
Gemini 2.5 Flash	81.2%	0.055	0.98 s
Gemini 2.5 Flash-Lite	69.7%	0.078	1.23 s
Gemini 2.5 Pro (pilota 50 img)	86.0%	0.035	6.2 s

Note sui risultati:

- **Gemini Pro è 5× più lento di Flash** e non offre vantaggi significativi in accuratezza (86% Pro vs 81% Flash su pilota di 50 immagini, con stessa lista di confusioni carattere): escluso dal run completo
- Gemini Flash è il **vincitore di rapporto accuratezza/costo** tra i modelli Gemini
- Flash-Lite è valida come opzione "low-cost" se il budget API è critico

3.3 Errori sistematici di Gemini Flash

Le confusioni di Gemini Flash sono **molto più contenute e prevedibili** di EasyOCR:

Carattere GT	Predetto come	Occorrenze
0 (zero)	O (lettera)	17

I	1	6
Q	O	3
0	G	3

Solo errori su caratteri **visivamente ambigui**, non errori di “forma del carattere” come in EasyOCR.

3.4 Limiti operativi

- **Latenza di rete:** 1 secondo nominale, ma può salire molto in caso di rete instabile
- **Dipendenza esterna:** la pipeline non funziona offline
- **Costo:** trascurabile per benchmark (~1 € per 735 immagini su Flash), ma non zero in produzione 24/7

4. fast-plate-ocr (Maggio 2026)

4.1 Cos'è

`fast-plate-ocr` è una libreria Python **specializzata per OCR di targhe automobilistiche**, open-source. Distribuisce modelli pre-addestrati via HuggingFace Hub, con inferenza ONNX Runtime ottimizzata.

Caratteristiche rilevanti:

- **Specializzata per targhe:** niente più problemi di delimitazione del contesto come EasyOCR, perché il modello assume che l'input sia già un crop di targa
- Inferenza **on-device via ONNX Runtime:** nessuna dipendenza dalla rete
- Modelli molto piccoli (2-30 MB)
- Installazione su Raspberry Pi 5: **una sola riga** (`pip install "fast-plate-ocr[onnx]"`), nessun problema di compilazione ARM64
- Modelli pre-addestrati su 40-65 paesi a seconda della variante

4.2 Modelli testati

Modello	Architettura	Color mode	Paesi training	Dimensione
---------	--------------	------------	----------------	------------

cct-xs-v1-global-model	CCT (Compact Convolutional Transformer) extra-small v1	RGB	65+	~2 MB
cct-s-v2-global-model	CCT small v2 (3× più training data)	RGB	65+	~30 MB
european-plates-mobile-vit-v2-model	MobileViT-2	GRAYSCALE	40+ Europa	~3 MB

4.3 Come funzionano: il vincolo del color mode

Un dettaglio tecnico rilevante: i modelli OCR specializzati come quelli di fast-plate-ocr sono **rigidi sul color mode di input**, perché viene fissato durante il training:

- I modelli cct-xs-v1 e cct-s-v2 sono addestrati su immagini **RGB a 3 canali**. L'input layer ha shape fissa `[1, 3, H, W]`. Passare un'immagine grayscale (1 canale) genera errore.
- Il modello european-plates-mobile-vit-v2 è addestrato su immagini **grayscale a 1 canale**. L'input layer ha shape `[1, 1, H, W]`. Passare un'immagine RGB genera errore.

Differenza con EasyOCR: EasyOCR è un framework completo con preprocessing flessibile interno, che converte automaticamente l'input nel formato necessario. fast-plate-ocr è più minimalista: se gli passi il **path del file** (`model.run("plate.png")`), la libreria gestisce internamente la conversione corretta. Se gli passi un **array numpy**, devi rispettare il formato richiesto dal modello.

Razionale della scelta in training:

- I CCT global sono addestrati su 65+ paesi inclusi paesi con targhe colorate (America Latina, alcuni paesi asiatici): RGB porta informazione utile
- Il modello "europeo" è addestrato solo su targhe europee, che sono uniformemente bianco-nero: grayscale è sufficiente, e il modello è più leggero/veloce

4.4 Risultati sul dataset European License Plates (735 immagini)

Modello	Exact match	Avg CER	Tempo medio (RPi5 CPU)
---------	-------------	---------	------------------------

cct-xs-v1-global-model	86.0%	0.029	21.3 ms
cct-s-v2-global-model	86.8%	0.028	127.6 ms
european-plates-mobile-vit-v2-model	80.0%	0.045	17.6 ms

Osservazioni chiave:

1. **Il modello “europeo dedicato” perde** contro i CCT global (80% vs 86%). Probabile causa: i CCT v2 sono stati addestrati su 3× più dati e includono comunque massivamente targhe europee. La specializzazione regionale non porta vantaggi qui.
2. **CCT-xs e CCT-s sono praticamente equivalenti in accuratezza** (86.0% vs 86.8%, differenza dentro il rumore statistico), ma **CCT-xs è 6× più veloce** (21 ms vs 128 ms) e **15× più piccolo** (2 MB vs 30 MB). Per pipeline real-time la scelta è chiara.
3. **Velocità impressionante:** 21 ms per crop su CPU RPi5, senza alcuna accelerazione. ~50 inferenze al secondo possibili.

5. Confronto finale tra tecnologie

5.1 Tabella sintetica

Confronto su dataset European License Plates, 735 immagini, niente post-processing applicato.

Tecnologia	Variante	Exact match	Avg CER	Tempo/crop	On-device	Dipendenze
EasyOCR	raw	16.7%	0.698	~1000 ms		nessuna
EasyOCR	grayscale	17.4%	0.697	~1239 ms		nessuna
Gemini 2.5 Flash-Lite	API	69.7%	0.078	1.23 s		rete + API key
Gemini 2.5 Flash	API	81.2%	0.055	0.98 s		rete + API key

Gemini 2.5 Pro (pilota)	API	86.0%	0.035	6.2 s		rete + API key
fast-plate-ocr cct-xs-v1	RGB	86.0%	0.029	21 ms		nessuna
fast-plate-ocr cct-s-v2	RGB	86.8%	0.028	128 ms		nessuna
fast-plate-ocr european-mobilevit-v2	GRAY	80.0%	0.045	18 ms		nessuna

5.2 Posizionamento delle tecnologie

Le tecnologie possono essere posizionate su due assi (accuratezza vs tempo per crop). La tabella seguente sintetizza il posizionamento operativo:

Tecnologia	Accuratezza	Velocità	Cluster operativo
fast-plate-ocr cct-s-v2	86.8%	128 ms	Alta accuratezza, velocità media
fast-plate-ocr cct-xs-v1	86.0%	21 ms	Alta accuratezza, alta velocità
Gemini 2.5 Pro	86.0%	6200 ms	Alta accuratezza, latenza elevata
Gemini 2.5 Flash	81.2%	980 ms	Buona accuratezza, dipendenza rete
fast-plate-ocr european-mobilevit-v2	80.0%	18 ms	Buona accuratezza, alta velocità
Gemini 2.5 Flash-Lite	69.7%	1230 ms	Bassa accuratezza, dipendenza rete
EasyOCR (grayscale)	17.4%	1239 ms	Insufficiente
EasyOCR (raw)	16.7%	1612 ms	Insufficiente

Il cluster “alta accuratezza, alta velocità” contiene un solo candidato: cct-xs-v1-global-model .

5.3 Confronto velocità

- **fast-plate-ocr cct-xs-v1**: 21 ms/crop, capacità teorica ~47 inferenze al secondo (47 FPS in OCR puro su CPU RPi5)
- **fast-plate-ocr eu-mobilevit-v2**: 18 ms/crop, capacità teorica ~56 inferenze al secondo (56 FPS)
- **fast-plate-ocr cct-s-v2**: 128 ms/crop, capacità teorica ~8 inferenze al secondo (7.8 FPS)
- **Gemini Flash**: ~1 s/crop, ~1 inferenza al secondo (1 FPS, limitato dalla rete)
- **EasyOCR**: ~1 s/crop, ~1 inferenza al secondo (1 FPS)
- fast-plate-ocr cct-xs-v1:
- fast-plate-ocr eu-mobilevit-v2:
- fast-plate-ocr cct-s-v2:
- Gemini Flash:
- EasyOCR:

In produzione la pipeline non sfrutta tutta la capacità su un singolo veicolo, ma la velocità di cct-xs-v1 permette di leggere ogni veicolo **decine di volte durante il suo transito davanti alla camera** (1-2 secondi tipici), abilitando un majority voting molto robusto.

Il salto di **50× in velocità** rispetto a EasyOCR è quello che cambia il paradigma operativo: con cct-xs a 21 ms, la pipeline può eseguire molte letture OCR per ogni veicolo in transito (assumendo 1-2 secondi di permanenza nel range utile della camera), abilitando un **majority voting su molte letture** che porta l'accuratezza reale ancora più in alto rispetto al singolo frame.

5.4 Confronto accuratezza per scenario

Targhe europee miste (dataset attuale):

1. cct-s-v2: 86.8%
2. cct-xs-v1: 86.0%
3. Gemini Pro: 86.0% (pilota)
4. Gemini Flash: 81.2%
5. european-mobilevit-v2: 80.0%
6. Gemini Flash-Lite: 69.7%
7. EasyOCR (grayscale): 17.4%

8. EasyOCR (raw): 16.7%

Atteso per targhe italiane (non ancora misurato sistematicamente su grande dataset):

- Per le tecnologie già specializzate o accurate (fast-plate-ocr, Gemini Flash) il risultato dovrebbe essere comparabile o migliore: le targhe italiane standard hanno formato rigido AA000AA con vincoli noti sui caratteri.

Nota sui vincoli del formato italiano post-1994: nelle targhe auto in formato AA000AA introdotte dal 1994 non vengono usate le lettere I, O, U, Q nelle posizioni lettera (per evitare confusioni con 1, 0, V, 0). Questo vincolo **vale solo per targhe auto post-1994** e non si applica a targhe storiche pre-1994 (con sigla provincia: MI, RO, FI, ecc.), targhe moto (formato AA12345), targhe di mezzi speciali (CC = Carabinieri, VF = Vigili del Fuoco, EI = Esercito, ecc.).

- Per EasyOCR si stima un miglioramento grazie a un post-processing pattern-based, ma comunque insufficiente per uso operativo.
-

6. Conclusioni e raccomandazioni operative

6.1 Tecnologia raccomandata

fast-plate-ocr con modello cct-xs-v1-global-model è la scelta consigliata per la pipeline NGS produttiva:

- **Accuratezza:** 86.0% exact match su targhe europee miste, equivalente ai migliori modelli (Gemini Pro) e ~5× superiore alla baseline EasyOCR
- **Velocità:** 21 ms per crop su CPU Raspberry Pi 5, ~50× più veloce di EasyOCR
- **Footprint:** solo 2 MB di modello, contro 70 MB di EasyOCR
- **Indipendenza:** 100% on-device via ONNX Runtime, nessuna dipendenza da rete o API esterne
- **Installazione:** una riga (`pip install "fast-plate-ocr[onnx]"`), zero problemi su ARM64
- **Specializzazione:** nato per targhe, niente problema di delimitazione del contesto

6.2 Strategia di pipeline

La velocità di cct-xs-v1 (21 ms/crop) abilita una nuova strategia operativa:

1. **Multi-frame reading:** ogni veicolo che passa davanti alla camera viene letto su 30-50 frame
2. **Majority voting:** la targa finale è quella più letta tra le predizioni

3. **Accuratezza reale attesa:** gli errori OCR a livello singolo frame (14%) sono prevalentemente non-sistematici, quindi il majority voting su decine di frame può portare l'accuratezza reale al **95%+**

6.3 Strategia di fallback (opzionale)

Pur essendo cct-xs-v1 sufficiente da solo, è possibile predisporre una strategia ibrida:

- **OCR primario:** fast-plate-ocr cct-xs-v1 (on-device, 21 ms)
- **Fallback opzionale:** Gemini Flash via API per casi a confidence bassa o pattern non riconoscibile
- **Costo trascurabile:** solo su casi difficili, ~0.001 € per chiamata, può essere disabilitato in caso di rete assente

6.4 EasyOCR

EasyOCR può essere **rimossa dalla pipeline produttiva**. Non offre vantaggi rispetto a fast-plate-ocr in nessuna dimensione:

- Accuratezza: 5× inferiore (17% vs 86%)
- Velocità: 50× più lenta (1000 ms vs 21 ms)
- Footprint: 35× più pesante (70 MB vs 2 MB)
- Specializzazione: nessuna

Resta utilizzata storicamente per validazione qualitativa e come riferimento di confronto in questo report.

6.5 Test futuri pianificati

1. **Benchmark fast-plate-ocr su targhe italiane:** appena disponibile un dataset italiano (in attesa di risposta da UniData per acquisto sample completo)
 2. **Validazione pipeline end-to-end** con v1n.hef detection + fast-plate-ocr OCR su video italiani realistici di camere semaforiche
 3. **Test post-processing italiano sopra fast-plate-ocr:** applicare il modulo `plate_postprocessing.py` (correzione I/O/Q/L/T nelle posizioni cifra, rimozione "IT" prefisso UE, cascata pattern moto/storica/moderna) ai casi falliti per quantificare il guadagno aggiuntivo
 4. **Integrazione in pipeline:** sostituzione tiny_yolov4 v1n per plate detection (già pianificata) + EasyOCR fast-plate-ocr per OCR
-

Appendice A — Note tecniche sulle inferenze

A.1 Color mode dei modelli fast-plate-ocr

Il color mode dell'input è fissato in fase di training e dichiarato nel file di config YAML che accompagna il modello ONNX. Quando si invoca `model.run(path_immagine)`, la libreria fast-plate-ocr legge l'immagine dal disco e converte automaticamente nel formato richiesto. Se invece si passa un array numpy direttamente, è responsabilità del chiamante fornire l'array nel formato corretto (3 canali RGB per cct-*, 1 canale grayscale per european-mobilevit-v2).

A.2 ONNX Runtime su ARM64

Tutti i modelli fast-plate-ocr girano via ONNX Runtime, che ha wheel pre-build per ARM64 e funziona stabilmente su Raspberry Pi 5. A differenza di PaddleOCR (testato in precedenza e risultato in segmentation fault su ARM64), ONNX Runtime non presenta problemi di compatibilità.

A.3 Dataset e setup riproducibilità

- Dataset European License Plates: 735 immagini in formato `.png`, `.jpg` e `.jpeg`
- Hardware test: Raspberry Pi 5 con 16 GB RAM, CPU only (Hailo-8 non utilizzato per OCR)
- Tutti gli script di benchmark sono nel repository `github.com/Bxster/ngs2026`
- Tempo totale di esecuzione del benchmark fast-plate-ocr full (3 modelli × 735 immagini): circa 3 minuti